

LibrIA — Cálculos, Equações e Fórmulas

Modelos Estático, Temporal e Híbrido

Repo: LibrIA

19 de março de 2026

Notação

- $x \in \mathbb{R}^D$: vetor de features estáticas (dimensão $D = \text{FEATURE_DIMENSION}$).
- T : número de estimadores (árvores) quando aplicável.
- K : número de classes.
- $\mathbf{X} \in \mathbb{R}^{L \times F}$: sequência temporal de comprimento L (sequence_length) e F features por frame (feature_size).
- $p(y = k \mid \cdot)$: probabilidade estimada para a classe k .

1. Extração e Normalização de Features

1.1 Modo *bounding_box* (42 features)

Dado o conjunto de 21 landmarks com coordenadas normalizadas (x_i, y_i) :

$$x'_i = x_i - \min_j x_j, \quad y'_i = y_i - \min_j y_j \quad (i = 1, \dots, 21).$$

O vetor de features é

$$\mathbf{f}_{\text{bbox}} = [x'_1, y'_1, \dots, x'_{21}, y'_{21}] \in \mathbb{R}^{42}.$$

1.2 Modo *wrist_relative* (63 features)

Defina o pulso (wrist) como $p_0 = (x_0, y_0, z_0)$. Para cada landmark i com $p_i = (x_i, y_i, z_i)$:

$$r_i = p_i - p_0, \quad d_i = \|r_i\|_2.$$

Se $d_{\max} = \max_i d_i > 0$, normaliza-se:

$$\tilde{r}_i = \frac{r_i}{d_{\max}}.$$

O vetor de features é o empilhamento:

$$\mathbf{f}_{\text{wrist}} = [\tilde{r}_1^\top, \tilde{r}_2^\top, \dots, \tilde{r}_{21}^\top] \in \mathbb{R}^{63}.$$

1.3 Espelhamento (mirror) usado em inferência/treino

- Se modo = *wrist_relative*: espelha eixo x :

$$\tilde{r}_{i,x} \leftarrow -\tilde{r}_{i,x}.$$

- Se modo = *bounding_box*: espelha como $x \leftarrow 1 - x$ (já normalizado no intervalo).

2. Modelo Estático (Random Forest)

Entrada $\mathbf{x} \in \mathbb{R}^D$.

Cada árvore t produz uma predição $h_t(\mathbf{x}) \in \{1, \dots, K\}$. A estimativa de probabilidade por classe é:

$$\hat{p}(k | \mathbf{x}) = \frac{1}{T} \sum_{t=1}^T \mathbf{1}\{h_t(\mathbf{x}) = k\}.$$

Decisão final:

$$\hat{y} = \arg \max_k \hat{p}(k | \mathbf{x}), \quad \text{confiança} = \max_k \hat{p}(k | \mathbf{x}).$$

Observação: se o objeto scikit-learn fornece $p(\cdot)$ diretamente, usa-se essa média internamente.

3. Arquitetura CNN Embutida (Estática)

Construção (entrada shape (P, C) — p.ex. $P = 21, C = 3$): - Camada Conv1D: filtros $f_1 = 16$, kernel $k = 3$, ativação ReLU. - Conv1D: $f_2 = 24$, kernel 3, ReLU. - MaxPool1D($s = 2$). - Conv1D: $f_3 = 32$, kernel 3, ReLU. - GlobalAveragePooling1D. - Dense(32), ReLU. - Dense(K), Softmax.

Operação de convolução 1D para um filtro $w \in \mathbb{R}^{k \times C}$ e bias b :

$$z_t = \sum_{j=0}^{k-1} w_j \cdot x_{t+j} + b, \quad a_t = \text{ReLU}(z_t) = \max(0, z_t).$$

Global average pooling sobre tempo:

$$g = \frac{1}{T'} \sum_{t=1}^{T'} a_t.$$

Camada densa final (logits):

$$z = Wg + b, \quad \hat{p} = \text{softmax}(z), \quad \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Função de perda usada (treino): sparse categorical crossentropy

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \log(\hat{p}_{y^{(n)}}(\mathbf{x}^{(n)})).$$

O otimizador: Adam (parâmetros padrão do Keras, não expostos como equação aqui).

4. Arquitetura CNN Temporal (Embeddable)

Entrada: $\mathbf{X} \in \mathbb{R}^{L \times F}$ com $L = \text{sequence_length}$ (tipicamente 30) e $F = \text{feature_size}$.

Camadas (conforme implementado): - Input(L, F) - Conv1D(24,3) ReLU - Conv1D(32,3) ReLU - MaxPool1D(2) - Conv1D(48,3) ReLU - GlobalAveragePooling1D - Dense(32) ReLU - Dense(K) softmax

As operações seguem as mesmas fórmulas de convolução, pooling e softmax da Secção 3.

5. Arquitetura LSTM (Modelo Temporal Host)

Entrada: $\mathbf{X} \in \mathbb{R}^{L \times F}$.

Camadas: - LSTM(128, return_sequences=True) - Dropout(0.3) - LSTM(64) - Dropout(0.3) - Dense(64) ReLU - Dense(K) softmax

Equações internas do LSTM (por passo temporal t ; vetores em negrito):

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Posteriormente, h_L (ou saída do último LSTM) é passada por Dense/ReLU e softmax para produzir $\hat{p}$; perda: mesma crossentropy como acima.

6. Regras de Inferência Híbrida e Arbitragem

Existem duas camadas de regras: (A) runtime embeddable `choose_embedded_prediction` e (B) `PredictionMerger` (host).

(A) Escolha no runtime embedded (regra declarativa): - Se existir previsão temporal válida e

$$\text{confidence}_{\text{temporal}} \geq \tau_{\text{temporal}} \quad \text{e} \quad \text{token}_{\text{temporal}} \in \mathcal{C}_{\text{prioridade}},$$

então escolher previsão temporal. - Senão, se

$$\text{confidence}_{\text{static}} \geq \tau_{\text{static}},$$

escolher previsão estática. - Senão, nenhum token é emitido.

(B) `PredictionMerger` (host) — comportamento formal: - Parâmetros: $\mathcal{C}_{\text{prioridade}}$ (temporal priority classes), τ_{temporal} , τ_{static} , t_{cooldown} . - Estado: $t_{\text{active_temporal}}$ (tempo até quando previsões estáticas são suprimidas). - Ao submeter evento temporal e com token k e confiança c : - Se $k \notin \mathcal{C}_{\text{prioridade}}$ OR $c < \tau_{\text{temporal}}$: ignorar (não emitir). - Caso contrário: emitir e e atualizar

$$t_{\text{active_temporal}} \leftarrow \max(t_{\text{active_temporal}}, t_{\text{end}}(e) + t_{\text{cooldown}}).$$

- Ao submeter evento estático s com confiança c e início t_{start} : - Se $c < \tau_{\text{static}}$: ignorar. - Se $t_{\text{start}} \leq t_{\text{active_temporal}}$: ignorar (bloqueado por janela temporal ativa). - Caso contrário: emitir s . Essas regras implementam desigualdades simples e atualizações de estado com máximo.

7. Métricas e Confiança

- Para modelos com probabilidade de saída: confiança = $\max_k \hat{p}(k \mid \cdot)$.
- Acurácia: $\frac{1}{N} \sum_n \mathbf{1}\{\hat{y}^{(n)} = y^{(n)}\}$.
- Durante treino registra-se *training/validation loss* e *accuracy* por época.

8. Observações de Implementação Relevantes

- Dimensões configuradas (em `config/settings.py`):

FEATURE_DIMENSIONS: bounding_box = 42, wrist_relative = 63.

Modo padrão: `wrist_relative`. Sequence_length típica: $L = 30$. - Quantização embedded: modelos Keras são exportados para TFLite int8 para deployment; fluxos de quantização aplicam mapeamento de ponto flutuante para representação inteira (não detalhado aqui, pois depende do calibrador e estatísticas de referência). - Random Forest usa parâmetros em MODEL_CONFIG (ex.: $n_{\text{estimators}} = 100$).

9. Referências de Códigos (onde as fórmulas são aplicadas)

- Pré-processamento / normalização: `utils/helpers.py` (`landmarks_to_relative`, `landmarks_to_bounding_box`).
- Modelos embarcados (CNN estático/temporal): `src/model_training/libras_embedded_cnn_trainer.py`, `src/model_training/libras_embedded_temporal_cnn_trainer.py` (arquiteturas Conv1D, pooling, GAP, Dense, Softmax).
- LSTM host: `src/model_training/libras_lstm_trainer.py` (camadas LSTM e configuração).
- RandomForest e pipeline estático: `src/model_training/libras_model_trainer.py` e artefato `model/model.pickle`.
- Lógica híbrida / arbitragem: `src/inference/prediction_merger.py` e `src/inference/libras_embedded_runtime.py`.

10. Conclusão

Este documento reúne as fórmulas essenciais para estudar a arquitetura: transformação de landmarks, operações convolucionais/recorrentes, função softmax/cross-entropy, estimativa de probabilidade do Random Forest e regras de arbitragem híbrida. Se quiser, eu: - gero o PDF compilado deste L^AT_EX; ou - salvo o .tex em `docs/` do repositório e abro um PR.